

EJERCICIOS Microcontroladores Y Microprocesadores

ARDUINO / C++

1. LED con tiempo aleatorio

Diseña un programa que encienda un LED en el pin D2 durante un tiempo aleatorio entre 200 ms y 10 s. Después debe apagarse durante 1 s y repetir el proceso indefinidamente.

```
#define LED 2

void setup(){
    Serial.begin(9600);
    pinMode(LED, OUTPUT);

    randomSeed(analogRead(A0));
}

void loop(){
    unsigned long rand_time = random(200,10001);

    digitalWrite(LED, HIGH);
    Serial.println("Led encendido por");
    Serial.println(rand_time);
    Serial.println("milisegundos");

    delay(rand_time);

    digitalWrite(LED, LOW);

    delay(1000);
}
```

2. LED PWM con dos intensidades

Controla un LED conectado al pin D5 con PWM. Debe funcionar al 20% de intensidad durante 1 s y luego pasar al 50% de intensidad durante 1 s, repitiéndose siempre.

```
#define LED 5

void setup(){
    Serial.begin(9600);
    pinMode(LED, OUTPUT);
}

void loop(){
    Serial.println("LED funcionando al 20%...");
    analogWrite(LED, 51);
    delay(1000);

    Serial.println("LED funcionando al 50%...");
    analogWrite(LED, 127);
}
```

```
    delay(1000);  
}
```

3. Sensor analógico + LED

Lee un sensor analógico conectado a A0. Si el valor leído es mayor que 10, activa una salida en D5. Muestra cada lectura por el monitor serie cada segundo

```
#define TMP36 A0  
#define LED 5  
  
void setup(){  
    Serial.begin(9600);  
    pinMode(TMP36, INPUT); // NO HARIA FALTA  
    pinMode(LED, OUTPUT);  
}  
  
void loop(){  
    Serial.println("Reseteando LED...");  
    digitalWrite(LED, LOW);  
    delay(500);  
  
    int TMP_value = analogRead(TMP36) - 10;  
    Serial.print("El valor leído por el sensor es:");  
    Serial.println(TMP_value);  
  
    if(TMP_value > 10)  
    {  
        digitalWrite(LED, HIGH);  
        Serial.println("LED encendido");  
        delay(500);  
    }  
    else  
    {  
        Serial.println("LED apagado");  
        digitalWrite(LED, LOW);  
        delay(500);  
    }  
}
```

4. LDR con PWM

Usa una LDR conectada a A0 y un LED PWM en D9. Cuanta menos luz haya, más brillo debe tener el LED. Cada 2 s, imprime el valor leído y el PWM aplicado.

```
#define LDR A0  
#define LED 9  
  
void setup(){  
    Serial.begin(9600);  
    pinMode(LED, OUTPUT);  
}
```

```

void loop(){
  int LDR_value = analogRead(LDR);

  Serial.print("Valor del LDR ");
  Serial.println(LDR_value);

  //invert values using map
  int pwm = map(LDR_value, 0, 1023, 255, 0);

  Serial.print("Encendiendo LED al ");
  Serial.println(pwm);
  analogWrite(LED, pwm);

  delay(2000);
}

```

5. LM35 temperatura

Lee un sensor LM35 en A1. Calcula la temperatura con:

$\text{temperatura} = \text{analogRead}(A1) * 5.0 * 100 / 1023;$

Si supera 30 °C, enciende un LED en D7. Imprime cada 3 s el valor analógico, la temperatura y el estado del LED.

```

#define LED 7
#define LM35 A1

void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
}

void loop(){
  Serial.print("Valor leído por el sensor:");
  Serial.println(analogRead(LM35));

  float temperatura = analogRead(A1) * 5.0 * 100 / 1023;
  Serial.print("Temperatura en grados:");
  Serial.println(temperatura);

  if (temperatura > 30)
  {
    Serial.println("Temp > 30. Encendiendo LED...");
    digitalWrite(LED, HIGH);
  }
  else
  {
    Serial.println("Temp < 30. Apagando LED...");
    digitalWrite(LED, LOW);
  }
  delay(3000);
}

```

```
}
```

6. Potenciómetro + ventilador

Lee un potenciómetro en A0 y controla la velocidad de un ventilador en D3 mediante PWM. La velocidad debe variar linealmente de 0 a 255. Imprime la velocidad cada 5 s.

```
#define FAN 3
#define POT A0

void setup(){
  Serial.begin(9600);
  pinMode(FAN, OUTPUT);
}

void loop(){
  int value = analogRead(POT);
  Serial.print("Valor leído por el potenciometro: ");
  Serial.println(value);

  int speed = map(value, 0, 1023, 0, 255);
  Serial.print("Velocidad del fan: ");
  Serial.println(speed);
  analogWrite(FAN, speed);

  delay(5000);
}
```

7. DHT22 + alarma

Usa un DHT22 conectado a D2. Lee temperatura y humedad cada minuto. Si la temperatura baja de 5 °C o la humedad supera el 90%, enciende un LED en D9. Imprime todo por Serial.

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT22

#define LED 9

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  if (isnan(humidity) || isnan(temperature))
  {
```

```

        Serial.println("Error: No se pudieron leer los
valores");
    }

    else
    {
        Serial.print("Humedad: ");
        Serial.println(humidity);

        Serial.print("Temperature: ");
        Serial.println(temperature);

        if(temperature < 5 || humidity > 90)
        {
            Serial.println("Encendiendo LED...");
            digitalWrite(LED, HIGH);
        }
        else
        {
            Serial.println("Apagando LED...");
            digitalWrite(LED, LOW);
        }
    }
    delay(60000);
}

```

8. Semáforo con pulsador

LED rojo en D2, amarillo en D3, verde en D4. Pulsador entre D5 y GND usando INPUT_PULLUP. El semáforo está en verde hasta que se pulse el botón. Entonces pasa a amarillo 3 s, rojo 10 s y vuelve a verde. El botón solo se atiende mientras está verde.

```

#define RED_LED 2
#define YELLOW_LED 3
#define GREEN_LED 4

#define BUTTON 5

void setup() {
    Serial.begin(9600);
    pinMode(RED_LED, OUTPUT);
    pinMode(YELLOW_LED, OUTPUT);
    pinMode(GREEN_LED, OUTPUT);
    pinMode(BUTTON, INPUT_PULLUP);
}

void loop() {
    Serial.println("Semaforo en verde");
    digitalWrite(RED_LED, LOW);
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);
}

```

```

    if((digitalRead(GREEN_LED) == HIGH) &&
digitalRead(BUTTON) == LOW)
    {
        digitalWrite(GREEN_LED, LOW);
        Serial.println("Semaforo en amarillo");
        digitalWrite(YELLOW_LED, HIGH);
        delay(3000);

        digitalWrite(YELLOW_LED, LOW);
        Serial.println("Semáforo en rojo");
        digitalWrite(RED_LED, HIGH);
        delay(10000);
        digitalWrite(RED_LED, LOW);
    }
}

```

9. LDR con memoria de mínimos y máximos

Lee una LDR en A0 cada 30 s durante 10 min. Guarda todas las lecturas en un array. Al final, muestra por Serial el valor mínimo y máximo. Si el mínimo es menor de 200, enciende un LED en D8 durante 5 s.

```

#define LED 8
#define LDR A0

void setup(){
    Serial.begin(9600);
    pinMode(LED, OUTPUT);
}

void loop(){
    digitalWrite(LED, LOW);
    int array[20];
    int min = 1023;
    int max = 0;
    int i = 0;
    while (i < 20)
    {
        array[i] = analogRead(LDR);
        if(array[i] > max)
        {
            max = array[i];
        }
        if(array[i] < min)
        {
            min = array [i];
        }

        i++;

        delay(300);
    }
}

```

```

Serial.print("Valor minimo del array: ");
Serial.println(min);

Serial.print("Valor maximo del array: ");
Serial.println(max);

if(min < 200)
{
    Serial.println("Encendiendo LED...");
    digitalWrite(LED, HIGH);
    delay(5000);
    digitalWrite(LED, LOW);
}
}

```

10. Sensor digital + LCD

Lee un sensor digital en D3. Muestra el valor en Serial y en una pantalla LCD. Si el valor leído supera un umbral que tú definas razonadamente, escribe "PELIGRO" tres veces por Serial. Repite cada segundo.

```

#include <LiquidCrystal_I2C.h>
#include <Wire.h>

#define PIR 3

LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
    Serial.begin(9600);

    pinMode(PIR, INPUT);

    lcd.init();
    lcd.backlight();
}

void loop() {
    int value = digitalRead(PIR);

    Serial.print("El valor leído por el sensor PIR es: ");
    Serial.println(value);

    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Valor:");
    lcd.setCursor(0, 1);
    lcd.print(value);

    if(value > 0)
    {
        Serial.println("PELIGRO");
    }
}

```

```
Serial.println("PELIGRO");
Serial.println("PELIGRO");
}
delay(1000);
```

RASPBERRY PI / PYTHON

1. Diseña un programa en Python para Raspberry Pi que haga parpadear un LED conectado a GPIO17:

- 1 segundo encendido
- 1 segundo apagado
- indefinidamente

Además:

- usar RPi.GPIO
- limpiar GPIO al terminar

```
import RPi.GPIO as GPIO
import time

LED = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)

try:
    while True:
        print("Encendiendo LED...")
        GPIO.output(LED, GPIO.HIGH)
        time.sleep(1)

        print("Apgando LED...")
        GPIO.output(LED, GPIO.LOW)
        time.sleep(1)

except KeyboardInterrupt:
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()
```

2. Un LED está conectado a GPIO18 y un pulsador a GPIO23. Cuando el usuario pulse el botón:

- el LED debe encenderse
- al soltarlo debe apagarse

El programa debe:

- mostrar el estado del botón por consola
- capturar KeyboardInterrupt

```
import RPi.GPIO as GPIO
import time
```

```

LED = 18
BUTTON = 23

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)
GPIO.setup(BUTTON, GPIO.IN)

try:
    while True:
        estado = GPIO.input(BUTTON)
        print ("El estado del pulsador es: ", estado)
        if estado == GPIO.HIGH:
            GPIO.output(LED, GPIO.HIGH)
        else:
            GPIO.output(LED, GPIO.LOW)

except KeyboardInterrupt:
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()

```

3. Diseña un programa que:

- lea un sensor PIR conectado a GPIO24
- si detecta movimiento:
 - imprimir "MOVIMIENTO DETECTADO"
 - encender un LED en GPIO25 durante 5 segundos

El sistema debe comprobar el sensor cada segundo.

```

import RPi.GPIO as GPIO
import time

PIR = 24
LED = 25

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)
GPIO.setup(PIR, GPIO.IN)

try:
    while True:
        estado = GPIO.input(PIR)
        if estado == GPIO.HIGH:
            print("MOVIMIENTO DETECTADO")
            GPIO.output(LED, GPIO.HIGH)
            time.sleep(5)
            GPIO.output(LED, GPIO.LOW)
            time.sleep(1)

except KeyboardInterrupt:
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()

```

4. Implementa el ejercicio clásico del examen:

- LED en GPIO6
- transmitir "SOS" indefinidamente en Morse
- usar constantes:
 - DOT
 - DASH
 - pausas
- crear función:
morse(symbols)
Además:
- limpiar GPIO al terminar

```
import RPi.GPIO as GPIO
import time

LED = 6
DOT = 0.2
DASH = 1
PAUSE = 0.2

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)

def morse(symbols):
    print(symbols)
    for i in symbols: //NO SE NECESITA HACER 'i = 0'
        if i == ".":
            GPIO.output(LED, GPIO.HIGH)
            time.sleep(DOT)
            GPIO.output(LED, GPIO.LOW)
        else if i == "-":
            GPIO.output(LED, GPIO.HIGH)
            time.sleep(DASH)
            GPIO.output(LED, GPIO.LOW)
        time.sleep(PAUSE)

try:
    while True:
        morse("...---...")
        time.sleep(1)

except KeyboardInterrupt:
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()

/*
Al hacer 'for i in symbols' ya no tenemos que hacer
'symbols[i]' porque i ya es el carácter
*/
```

5. Simula un potenciómetro usando teclado:

- pedir al usuario un valor entre 0 y 100
- convertirlo a PWM 0-255
- controlar brillo de un LED conectado a GPIO12 mediante PWM

```
import RPi.GPIO as GPIO
import time

LED = 12

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)

pwm = GPIO.PWM(LED, 100)
pwm.start(0)

try:
    while True:
        value = float(input("Introduce un valor entre 0 y 100:"))

        if value < 0 or value > 100:
            print("Valor fuera de rango")
        else:
            print("Valor de led:", value, "%")
            pwm.ChangeDutyCycle(value)

except KeyboardInterrupt:
    pwm.stop();
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()

/*
1. Creamos el objeto 'pwm', es decir, "activar modo PWM en ese
pin" (utilizamos 'start()' y 'stop()')

2. Con 'input()' leemos el valor dado por el usuario y con un
'float' antes de la llamada, casteamos

3. Cambiamos el porcentaje con 'ChangeDutyCycle()' de encendido
*/
```

6. Usa una pantalla LCD I2C y un sensor de temperatura para mostrar temperatura cada 2 segundos y , si supera los 30 °C:

- mostrar "ALERTA" en LCD
- escribirlo también por consola

```
from RPLCD.i2c import CharLCD
//import RPi.GPIO as GPIO
import time

lcd = CharLCD('PCF8574', 0x27)
DHT_SENSOR = Adafruit_DHT.DHT22
SENSOR_PIN = 3
```

```

//GPIO.setmode(GPIO.BCM)
//GPIO.setup(SENSOR_PIN, GPIO.IN)

try:
    while True:
        lcd.clear()
        h, temperature = Adafruit_DHT.read_retry(DHT_SENSOR,
        SENSOR_PIN)
        if temperature is not None:
            print("Temperatura: ", temperature)
            if temperature > 30:
                print("Temperatura superior a 30 grados. Mostrando
                alerta...")
                lcd.text("ALERTA", 2)
                print("ALERTA")
            else:
                print("Error: Valor invalido")

except KeyboardInterrupt:
    lcd.clear()
    //GPIO.cleanup()

/*
- No hacen falta las librerías de GPIO porque se supone que
Adafruit del sensor gestiona directamente los pines en su
interior
- is None = isnan() en C++
*/

```

7. Controla un servo en GPIO18:

- moverlo:
 - 0º
 - 90º
 - 180º
- esperar 2 segundos entre posiciones
- repetir indefinidamente

```

import RPi.GPIO as GPIO
import time

SERVO_PIN = 18

GPIO.setmode(GPIO.BCM)
GPIO.setup(SERVO_PIN, GPIO.OUT)

pwm = GPIO.PWM(SERVO_PIN, 50)

def setAngle(angle):
    return angle / 18 + 2 // entendemos que 12% es el 0

try:
    pwm.start(0)

```

```

while True:
    pwm.ChangeDutyCycle(setAngle(0))
    time.sleep(2)
    pwm.ChangeDutyCycle(setAngle(90))
    time.sleep(2)
    pwm.ChangeDutyCycle(setAngle(180))
    time.sleep(2)

except KeyboardInterrupt:
    pwm.stop()
    GPIO.cleanup()

```

8. Diseña un programa para HC-SR04:

- medir distancia cada segundo
- mostrarla por consola
- si es menor de 10 cm:
- encender LED rojo

```

LED = 3
ECHO = 23
TRIGGER = 24

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(TRIGGER, GPIO.OUT)

try:
    while True:
        GPIO.output(TRIGGER, True)
        time.sleep(0.00001)
        GPIO.output(TRIGGER, False)

        while GPIO.input(ECHO) == 0:
            pulse_start = time.time()

        while GPIO.input(ECHO) == 1:
            pulse_end = time.time()

        pulse_duration = pulse_end - pulse_start
        distancia = pulse_duration * 17150 // 34300/2 = 17150
        distancia = round(distancia, 2)
        print("Distancia: ", distancia)

        if distancia < 10:
            print("Distancia < 10 cm. Encendiendo LED...")
            GPIO.output(LED, GPIO.HIGH)
        else:
            GPIO.output(LED, GPIO.LOW)
            time.sleep(1)

except KeyboardInterrupt:

```

```
GPIO.output(LED, GPIO.LOW)
GPIO.cleanup()
```

9. Usa una LDR:

- si hay poca luz:
 - encender LED
- si hay mucha luz:
 - apagar LED

Además:

- imprimir valores cada 5 segundos

```
import RPi.GPIO as GPIO
import time

LED = 3
LDR = 4

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)
GPIO.setup(LDR, GPIO.IN)

try:
    while True:
        value = GPIO.input(LDR)
        print("Valor obtenido del LDR: ", value)
        if value == GPIO.HIGH:
            GPIO.output(LED, GPIO.HIGH)
        else:
            GPIO.output(LED, GPIO.LOW)
        time.sleep(5)

except KeyboardInterrupt:
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()
```

10. Diseña un sistema con:

- PIR
 - buzzer
 - LED rojo
- Si se detecta movimiento:
- activar buzzer
 - encender LED
 - mostrar 'PELIGRO' por consola 3 veces.

```
import RPi.GPIO as GPIO
import time

LED = 3
BUZZER = 4
PIR = 5

GPIO.setmode(GPIO.BCM)
```

```
GPIO.setup(LED, GPIO.OUT)
GPIO.setup(BUZZER, GPIO.OUT)
GPIO.setup(PIR, GPIO.IN)

try:
    while True:
        pir_value = GPIO.input(PIR)
        if pir_value == GPIO.HIGH:
            GPIO.output(BUZZER, GPIO.HIGH)
            GPIO.output(LED, GPIO.HIGH)
            print("PELIGRO")
            print("PELIGRO")
            print("PELIGRO")
        else:
            GPIO.output(BUZZER, GPIO.LOW)
            GPIO.output(LED, GPIO.LOW)
            time.sleep(1)

except KeyboardInterrupt:
    GPIO.output(LED, GPIO.LOW)
    GPIO.output(BUZZER, GPIO.LOW)
    GPIO.cleanup()
```

TEORÍA

1. Explique las diferencias entre Raspberry Pi y Arduino, indicando:

- sistema operativo
- tiempo real
- capacidad de procesamiento
- GPIO

Las principales diferencias entre Arduino y Raspberry son que Arduino es una placa basada en un microcontrolador, mientras que la Raspberry Pi es una placa basada en un microprocesador. Esto, en términos de SO, significa que los microcontroladores, en este caso, la placa Arduino, no utiliza un SO complejo, en cambio, la Raspberry, necesita instalar un SO como Raspbian (SO basado en Debian).

En cuanto a su capacidad de procesamiento, Arduino está orientado a labores específicas que ejecuten una sola tarea a la vez como, por ejemplo, “cerebro” de un regulador de temperatura. Raspberry Pi está diseñado para poder ejecutar más tareas y programas a la vez bajo un sistema operativo más complejo, se acercaría más al comportamiento de un ordenador. Esto hace que el Arduino sea mas limitado en cuanto a procesos e instrucciones que la placa Raspberry Pi.

En cuanto al tiempo real, Arduino ofrece un comportamiento más preciso (microsegundos) que Raspberry ya que su procesamiento es determinista, es decir, responde a los estímulos del hardware de forma inmediata. En cambio, Raspberry, al gestionar un SO más complejo, debe gestionar múltiples procesos y tareas en segundo plano que producen retrasos.

Hay una gran diferencia de pines, si comparamos Arduino UNO y Raspberry Pi 3, los más estándares, nos daremos cuenta de que la Raspberry cuenta con 40 pines GPIO, entre ellos se gestionan los pines GND (tierra), alimentación y los pines de comunicación, mientras que

Arduino UNO cuenta con 20, 14 pines digitales y 6 pines analógicos, esto significa que Arduino, a diferencia de Raspberry, viene con sistema ADC integrado y varios pines con hardware PWM. Además, la alimentación entre ambas placas cambia, mientras que Arduino trabaja a 5V, Raspberry trabaja a 3.3V.

2. ¿Qué es PWM y para qué se utiliza? Explique:

- ciclo de trabajo
- frecuencia
- relación con:
 - LEDs
 - motores
 - servos

PWM (Pulse Width Modulation) es una técnica incluida en hardware del Arduino que sirve para poder, como su propio nombre indica, modular el ancho de los pulsos de la señal eléctrica.

El ciclo de trabajo (duty cycle) representa el porcentaje de tiempo que la señal permanece en HIGH durante un ciclo completo. Por ejemplo, un duty cycle del 50% significa que la señal está encendida la mitad del tiempo y apagada la otra mitad, lo que nos dará una sensación de una intensidad del 50%.

La frecuencia indica cuántos ciclos PWM se producen por segundo y se mide en hercios (Hz). Esta técnica nos es útil para poder controlar tiempos e interrupciones e incluso simular respuestas analógicas desde entradas digitales. Por ejemplo, un LED en una entrada digital responde a 1 (HIGH / encendido) y 0's (LOW / apagado). Mediante la tecnología PWM podemos modificar el tiempo de estas señales para que sean cortas y seguidas, es decir modificar el ciclo de trabajo de estos ciclos, como parpadeos, lo que, para el ojo humano, se interpretará como diferentes intensidades del LED.

3. Explique el funcionamiento de:

- digitalWrite
- digitalWrite
- analogRead
- analogWrite

Las siguientes funciones se utilizan en Arduino para leer o escribir de nuestros pines de comunicación:

- digitalWrite: Utilizamos esta función para leer la respuesta (1-HIGH o 0-LOW) de un pin digital INPUT
- digitalWrite: Utilizamos esta función para escribir el comportamiento (1-HIGH o 0-LOW) de un pin digital OUTPUT.
- analogRead: Utilizamos esta función para leer la respuesta de un pin analógico INPUT. Esta función mediante tecnología ADC es capaz de convertir el comportamiento analógico en un valor digital de rango, normalmente, 0-1023.
- analogWrite: Utilizamos esta función para escribir el comportamiento de un pin "analógico" (pines con PWM) OUTPUT. Esta función mediante tecnología PWM es capaz de simular electrónicamente un comportamiento analógico de rango, normalmente, 0-255, mediante una señal digital PWM.

4. Explique UART, SPI e I2C. Indicando:

- número de hilos
- velocidad
- ventajas
- inconvenientes

UART, SPI e I2C son todos protocolos de comunicación utilizados para intercambiar información entre microcontroladores y dispositivos externos.

UART (Universal Asynchronous Receiver Transmitter) es una comunicación serie asíncrona que utiliza dos hilos, RX y TX. No necesita señal de reloj y es sencilla de implementar, aunque solo permite la comunicación entre dos dispositivos y tiene menor velocidad que SPI.

SPI (Serial Peripheral Interface) es una comunicación serie síncrona que utiliza normalmente cuatro hilos: MOSI, MISO, SCK y CS. Destaca por su elevada velocidad y por permitir transmitir y recibir datos simultáneamente, lo que se denomina una comunicación full-dúplex. Esto requiere más cableado y un mayor número de conexiones ya que cada dispositivo necesita una línea CS adicional.

I2C (Inter-Integrated Circuit) es una comunicación serie síncrona que utiliza únicamente dos hilos, SDA para los datos y SCL para el reloj. Su principal ventaja es que permite conectar varios dispositivos utilizando el mismo bus, reduciendo considerablemente el cableado, aunque su velocidad es inferior a la de SPI y su configuración más compleja.

5. Describa el funcionamiento de una LDR. Explique:

- divisor de tensión
- conversión resistencia → voltaje
- uso con ADC

Una **LDR (Light Dependent Resistor)** es una resistencia cuyo valor varía en función de la cantidad de luz que recibe. Cuando aumenta la iluminación, su resistencia disminuye, mientras que en condiciones de oscuridad su resistencia aumenta.

Habitualmente se conecta formando un divisor de tensión junto con una resistencia fija, de manera que las variaciones de resistencia se transforman en variaciones de voltaje. Este voltaje puede conectarse a una entrada analógica de Arduino, donde el conversor ADC lo transforma en un valor digital comprendido normalmente entre 0 y 1023 mediante la función `'analogRead()'`

6. Explique qué es INPUT_PULLUP. Indique:

- funcionamiento interno
- ventajas
- comportamiento HIGH/LOW

INPUT_PULLUP es un modo de configuración de los pines digitales de Arduino que activa una resistencia pull-up interna conectada entre el pin y la alimentación. Esta resistencia evita que el pin quede flotante cuando no recibe ninguna señal externa, eliminando lecturas erráticas. Su principal ventaja es que no requiere una resistencia externa adicional, simplificando el circuito. Cuando se utiliza INPUT_PULLUP, la lógica de funcionamiento queda invertida: el pin se encuentra normalmente en estado HIGH y pasa a estado LOW cuando se conecta a masa, por ejemplo, al pulsar un botón.

7. Describa la estructura básica de un programa Arduino:

- setup
- loop
- bibliotecas
- variables globales

Un programa Arduino está formado por bibliotecas, variables globales y dos funciones principales llamadas `setup()` y `loop()`. Las bibliotecas se incluyen al inicio del programa mediante la directiva `#include` y permiten utilizar funciones desarrolladas previamente. Las variables globales se declaran fuera de las funciones y pueden utilizarse desde cualquier parte del programa. La función `setup()` se ejecuta una única vez al iniciar el microcontrolador y se emplea para configurar pines, comunicaciones y periféricos. La función `loop()` se ejecuta de forma repetitiva mientras Arduino permanezca encendido y contiene el código principal de funcionamiento del programa.

8. Explique cómo funciona un sensor PIR. Indique:

- qué detecta
- salida digital
- aplicaciones típicas

Un sensor **PIR (Passive Infrared Sensor)** es un dispositivo capaz de detectar cambios en la radiación infrarroja emitida por cuerpos calientes, como personas o animales. Cuando detecta movimiento dentro de su zona de alcance, genera una señal digital en su salida. Normalmente esta salida toma el valor HIGH cuando detecta movimiento y LOW cuando no detecta actividad. Este tipo de sensores se utiliza habitualmente en sistemas de alarma, iluminación automática, control de presencia y aplicaciones de domótica.

9. Explique el funcionamiento de un display LCD I2C:

- dirección I2C
- SDA
- SCL
- ventajas frente a LCD paralelo

Un display **LCD I2C** es un monitor LCD convencional al que se le añade un adaptador que permite utilizar el protocolo de comunicación I2C. Cada dispositivo conectado al bus I2C dispone de una dirección única que permite identificarlo. La comunicación se realiza mediante dos líneas: SDA, encargada de transmitir los datos, y SCL, encargada de proporcionar la señal de reloj. La principal ventaja de los displays LCD I2C frente a los LCD conectados en modo paralelo es que requieren únicamente dos pines del microcontrolador, reduciendo el cableado y facilitando la conexión de varios dispositivos al mismo tiempo.

10. Explique memoria RAM, ROM, EEPROM y Flash en microcontroladores Atmel usados en Arduino.

Los microcontroladores Atmel utilizados en Arduino disponen de distintos tipos de memoria con funciones específicas. La memoria **RAM** es una memoria volátil utilizada para almacenar variables y datos temporales durante la ejecución del programa; su contenido se pierde al apagar el dispositivo. La memoria **ROM** es una memoria de solo lectura que almacena información fija del sistema. La memoria **EEPROM** es una memoria no volátil que permite guardar datos que deben conservarse incluso cuando el dispositivo se apaga, como configuraciones o parámetros. Finalmente, la memoria **Flash** es una memoria no volátil donde se almacena el programa o sketch de Arduino que se ejecutará cada vez que el microcontrolador se inicie.